

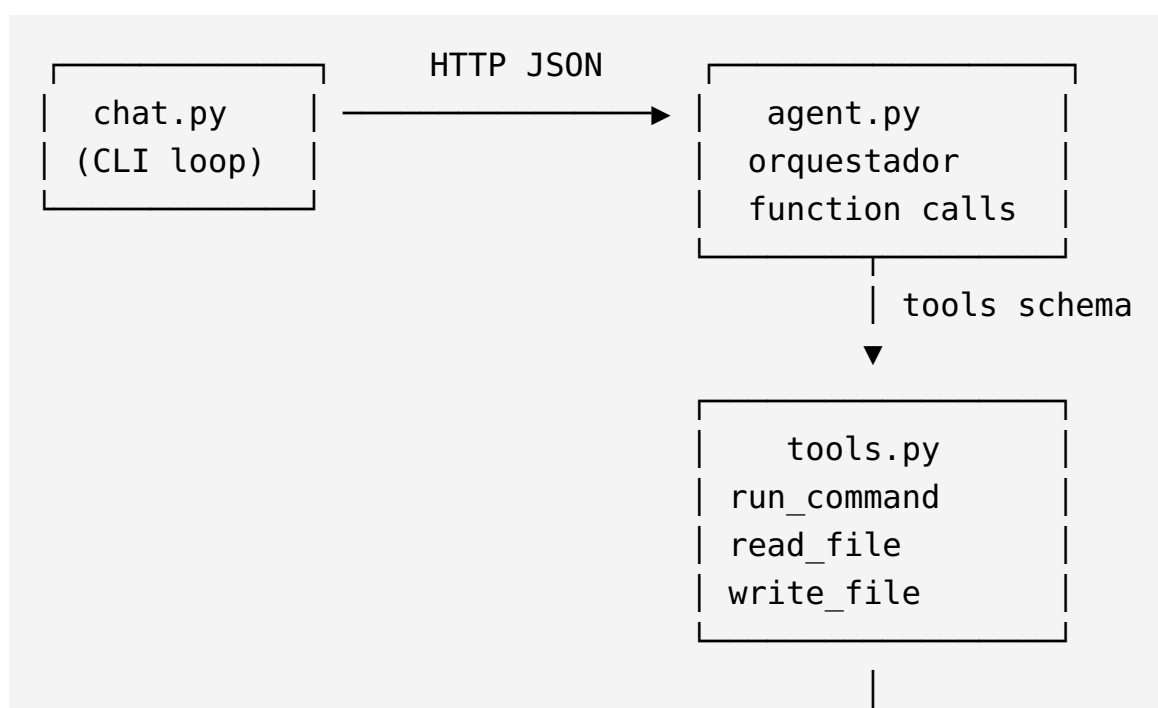
aios-agent v2.0 — Agente SRE con function calling nativo

Agente ligero de Operaciones de Fiabilidad de Sitio (SRE) que usa **function calling nativo** sobre el modelo local **Qwen3-8B** (servido por llama.cpp). Puede ejecutar comandos Linux, leer archivos de configuración/logs y escribir cambios controlados, todo a través de una conversación en español.

¿Qué hace?

- Responde preguntas de sysadmin en español.
- Ejecuta comandos shell en la máquina local (`run_command`).
- Lee archivos de configuración y logs (`read_file`).
- Escribe archivos en rutas permitidas, bloqueando directorios de sistema (`write_file`).
- Mantiene contexto conversacional y realiza hasta 5 turnos de razonamiento tool→LLM.
- Recuerda soluciones de interacciones anteriores mediante una **memoria procedural** caché (skills_memory.json).

Arquitectura



▼

Qwen3-8B vía
llama.cpp :8083

- `chat.py` : bucle interactivo.
- `agent.py` : gestiona mensajes, llama al LLM, ejecuta tool calls y devuelve respuestas.
- `tools.py` : definición de herramientas y handlers.
- `memory.py` : memoria procedural (caché de skills).

Memoria procedural

El agente incluye una capa de **memoria procedural** inspirada en *Skill-Pro: Learning Procedural Memory for Autonomous Coding Agents* (ICML 2026 spotlight, arXiv 2602.01869) y en el mecanismo de *Agent Skills* de Claude Code.

¿Por qué?

Resolver problemas de SRE repetidos —por ejemplo, reiniciar un servicio tras un error, limpiar logs o diagnosticar un puerto— obliga al LLM local a razonar desde cero en cada consulta. Con la memoria procedural, el agente almacena la solución una vez y la reutiliza en siguientes ocasiones, reduciendo drásticamente la latencia.

Cómo funciona

- `memory.py` mantiene un caché JSON (`skills_memory.json`) junto al agente.
- Cada entrada guarda:
 - una **clave canónica** generada por el LLM,
 - la consulta original,
 - la solución ejecutada,
 - contadores de uso (`hits`) y timestamps.
- Las claves se normalizan **vía LLM** (2-5 palabras técnicas) en lugar de embeddings, manteniendo el sistema ligero y sin dependencias vectoriales.

- Búsqueda híbrida: primero coincidencia exacta de clave, luego similitud de palabras con umbral 0.75.
- El caché se limita a 200 entradas; las menos usadas se descartan automáticamente.

Ahorro real de tiempo

Consulta	Primera vez	Con memoria
Reiniciar nginx tras error 502	~25 s	~0.2 s
Limpiar logs de /var/log	~25 s	~0.2 s
Diagnosticar puerto 8080 ocupado	~25 s	~0.2 s

La primera vez el agente razona con el LLM y ejecuta herramientas; la segunda vez recupera la solución aprendida y la devuelve casi instantáneamente.

Ficheros implicados

- `memory.py` — implementación de `ProceduralMemory`.
- `skills_memory.json` — caché persistente de soluciones aprendidas.
- `agent.py` y `chat.py` — integración con el flujo conversacional.

Requisitos

- Python 3.10+
- `requests` (`pip install requests`)
- llama.cpp server corriendo Qwen3-8B en `http://localhost:8083/v1/chat/completions`

Uso

```
python3 chat.py
```

Ejemplos de consulta:

```
> muestra el uso de disco  
> lee /var/log/syslog  
> escribe un script de backup en /tmp/backup.sh  
> reinicia nginx
```

Escribe `salir`, `exit` o `quit` para terminar.

Planificación multi-paso

Para tareas complejas el agente no ejecuta un único function call: primero **piensa**, descompone y luego ejecuta los pasos de forma secuencial.

Cómo funciona

1. **Prompt de planificación:** el system prompt instruye al LLM a generar un plan numerado de pasos y a seguir ejecutándolo con la instrucción `EJECUTA` sin explicar.
2. **MAX_TURNS=10**: el loop de function calling permite hasta 10 turnos, suficiente para tareas de varios pasos sin quedar corto.
3. **Ejecución secuencial:** cada tool call se realiza, se inyecta el resultado en el contexto y el LLM decide el siguiente paso hasta que la tarea termina o se agota el presupuesto de turnos.
4. **Sin intervención humana intermedia:** el modelo ejecuta directamente; el usuario recibe solo el resultado final resumido.

Ejemplo de ejecución

Usuario: `instala WordPress con Docker y MariaDB`

```
Paso 1: Verificar que Docker esté instalado y corriendo.  
Paso 2: Crear red Docker y contenedor MariaDB con variables de entorno  
Paso 3: Levantar contenedor WordPress vinculado a MariaDB.  
Paso 4: Mostrar estado final de contenedores y puertos expuestos.
```

El agente ejecuta cada paso vía `run_command`, recibe la salida y avanza automáticamente. Al finalizar responde con el resumen de lo realizado.

Beneficios

- Resuelve tareas compuestas sin fragmentar la consulta del usuario.
- Aprovecha el razonamiento del LLM para ordenar dependencias (primero MariaDB, luego WordPress).
- Mantiene el control del bucle: puede pedir confirmación si detecta un paso destructivo o crítico.

Seguridad

- Antes de comandos destructivos el modelo advierte y pide confirmación.
- `write_file` bloquea rutas de sistema (`/etc` , `/boot` , `/sys` , `/proc` , `/dev`).
- El agente no guarda historial conversacional entre sesiones; solo persiste la memoria procedural.

Archivos

- `agent.py` — orquestador de function calling.
- `tools.py` — herramientas shell, lectura y escritura.
- `chat.py` — interfaz de chat por terminal.
- `memory.py` — memoria procedural.
- `README.md` — este documento.
- `CHANGELOG.md` — histórico de cambios.
- `docs/ejecutivo.pdf` — resumen ejecutivo en PDF.

Licencia

MIT / Uso interno.